

Python

Summary Session 4



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Экспресс-опрос

?

Что такое цикл?

?

Как выглядит синтаксис цикла while?



Цикл

Это конструкция в программировании, которая позволяет повторять выполнение блока кода (тела цикла) несколько раз, пока выполняется определённое условие или пока не пройдётся по всем элементам последовательности.



Итерация

Это одно выполнение тела цикла. Итерации происходят до тех пор, пока выполняется условие цикла.



Цикл while

Это цикл с предусловием, который выполняет блок кода до тех пор, пока условие остаётся истинным.

Синтаксис цикла while



```
while условие: # условие цикла
```

```
    # блок кода или тело цикла
```


Синтаксис цикла while



Условие: Это выражение, которое проверяется перед каждой итерацией. Если оно истинно (True), выполняется блок кода. Как только условие становится ложным (False), цикл завершает свою работу.



Блок кода: Код, который будет выполняться на каждой итерации, пока условие истинно. После выполнения блока кода интерпретатор всегда возвращается к проверке условия.

Пример



```
i = 1  
while i <= 5:  
    print(i)  
    i += 1
```

Объяснение



Переменная i инициализируется значением 1



Цикл продолжается до тех пор,
пока значение переменной i меньше или равно 5



Внутри цикла переменная увеличивается на 1 с каждой итерацией с помощью оператора $i += 1$



Как только i становится больше 5, цикл завершается

Экспресс-опрос

?

Что такое оператор break?

?

Для чего необходимо использование break с пользовательским вводом?



Оператор break

Используется для принудительного завершения выполнения цикла.

Пример



```
i = 1
while i <= 10:
    print(i)
    if i == 5:
        break # Прерывание цикла, когда i станет равно 5
    i += 1
```

Использование break с ПОЛЬЗОВАТЕЛЬСКИМ ВВОДОМ

Оператор break может быть полезен для выхода из цикла, когда программа ожидает ввода от пользователя и требуется завершить цикл при определённых условиях.

Пример



```
while True:

    user_input = input("Введите 'exit', чтобы завершить цикл: ")

    if user_input == "exit":

        break

    print("Вы ввели:", user_input)
```


Экспресс-опрос

?

Что такое оператор continue?

?

Приведите пример использования continue.



Оператор continue

В циклах используется для пропуска оставшейся части кода текущей итерации и перехода к следующей итерации цикла.

Синтаксис



```
while условие:

    # код до continue

    if условие_для_continue:

        continue # пропуск оставшегося кода в этой итерации

    # код, который будет пропущен, если сработает continue
```

Пример использования continue



```
i = 0

while i < 5:

    i += 1

    if not i % 2:

        continue # Пропускаем итерацию, когда i четное

    print(i)
```

Синтаксис



```
result = 1

while True:

    user_input = input("Введите число для перемножения: ")

    if user_input == "0":

        print("Пропуск итерации")

        continue # Пропускаем оставшуюся часть текущей итерации

    if user_input == "exit":

        print("Выход из программы")

        break # Прерывание цикла

    result *= int(user_input)

print("Результат перемножения:", result)
```

Экспресс-опрос

?

Что такое бесконечный цикл?

?

Приведите пример бесконечного цикла.



Бесконечный цикл

Это цикл, который никогда не завершает своё выполнение, потому что его условие всегда остаётся истинным.

Пример



```
while True:  
    print("Этот цикл будет выполняться бесконечно")
```


Пример



```
while True:
    user_input = input("Введите 'stop', чтобы завершить цикл: ")
    if user_input == "stop":
        print("Цикл завершён.")
        break # Прерывание цикла, если пользователь ввёл 'stop'
```



Бесконечный цикл

Может возникнуть случайно, если неправильно прописано условие завершения.

Пример



```
i = 0
```

```
while i < 10:
```

```
    print(i)
```

Пропущено увеличение i, поэтому условие всегда истинно, и цикл бесконечен

Пример



```
i = 0
```

```
while i < 10:
```

```
    print(i)
```

```
    i -= 1
```

```
# i меняется в другую сторону, не приближаясь к условию выхода
```

Экспресс-опрос

?

Что такое конструкция while/else?



Конструкция while/else

Позволяет выполнить блок кода `else` после завершения цикла `while`, при нормальном завершении цикла, то есть без использования оператора `break`.

Синтаксис



```
while условие:
    # блок кода while
else:
    # блок кода else
```

Пример



```
i = 1

while i <= 5:

    print(i)

    i += 1

else:

    print("Цикл завершён без прерываний.")
```


Пример



1

```
result =  
num = 1  
  
while num < 6:  
  
    user_input = input("Введите " + str(num) + "-е число для перемножения: ")  
    num += 1  
  
    if user_input == "0":  
  
        print("Некорректные данные. Выход из программы")  
  
        break # Прерывание цикла  
  
    result *= int(user_input)  
  
else:  
  
    print("Цикл завершён без прерываний.")  
  
print("Результат перемножения:", result)
```



continue в while/else

Оператор `continue` не прерывает цикл, а только пропускает часть итерации, поэтому блок `else` всё равно выполнится.

Пример



```

result = 1
num = 1
while num < 6:
    user_input = input("Введите " + str(num) + "-е число для перемножения: ")
    num += 1
    if user_input == "0":
        print("Некорректные данные. Выход из программы") /
        break # Прерывание цикла
    if int(user_input) > 1000:
        print("Слишком большое число. Пропуск итерации")
        continue # Пропускаем оставшуюся часть текущей итерации
    result *= int(user_input)
else:
    print("Цикл завершён без прерываний.")
print("Результат перемножения:", result)

```

Экспресс-опрос

?

Что такое модуль random?



Модуль random

Это стандартный модуль Python, который предоставляет функции для генерации случайных чисел, а также для работы со случайным выбором элементов из коллекций.

Генерация случайного числа с плавающей запятой



```
import random
```

```
# Генерируем случайное число от 0.0 до 1.0
```

```
print(random.random())
```

Генерация случайного целого числа



```
import random
```

```
# Генерируем случайное целое число от 1 до 10 (включая обе границы)
```

```
print(random.randint(1, 10))
```

Синтаксис



```
import random
```

```
# Генерируем случайное целое число от 1 до 10 (не включая число 10) и с шагом 2 (только по нечетным числам)
```

```
print(random.randrange(1, 10, 2))
```


Когда использовать random



Симуляции: Игры, моделирование случайных событий.



Генерация тестовых данных: Создание случайных чисел или строк для тестирования алгоритмов.



Программы, требующие случайности: Лотереи, случайное распределение задач.



Перемешивание данных: Например, случайное перемешивание списка студентов для презентации или задания.

Экспресс-опрос



Что такое строки?



Строки

Это упорядоченные последовательности символов, заключённых в одинарные или двойные кавычки, а также в тройные кавычки (как одинарные, так и двойные).

Пример неизменяемости строки



```
s = "Hello"
print(s + " world!")
print(s)
```

"Изменение" содержимого строки



```
s = "Hello"  
  
s = s + " world!"  
  
print(s)
```

Экспресс-опрос

?

Что такое кодировка ASCII?



Кодировка

Это способ преобразования символов в числа, которые компьютер может хранить и обрабатывать.



Кодировка ASCII

Это одна из первых и простейших кодировок, разработанная для представления текстовых символов в компьютерах.

Характеристики кодировки ASCII

7 бит

Символы

Простота и совместимость

ASCII использует 7 бит для кодирования символов.

0-31: Управляющие символы (например, символ новой строки, возврат каретки, сигнал окончания текста).

ASCII был создан для работы с английским алфавитом, цифрами и основными символами.

Это позволяет закодировать 128 символов ($2^7 = 128$).

Символы представлены числами от 0 до 127.

32-126: Печатные символы, включая буквы, цифры, пунктуацию и другие символы.

Он широко используется в компьютерах, сетевых системах и других устройствах, обеспечивая базовый уровень кодирования для текстовых данных.

Таблица символов ASCII

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Расширения ASCII

символ	10-й код	2-й код	символ	10-й код	2-й код	символ	10-й код	2-й код	символ	10-й код	2-й код
Ъ	128	10000000		160	10100000	А	192	11000000	а	224	11100000
Г	129	10000001	Ѹ	161	10100001	Б	193	11000001	б	225	11100001
,	130	10000010	ѹ	162	10100010	В	194	11000010	в	226	11100010
г	131	10000011	Ј	163	10100011	Г	195	11000011	г	227	11100011
„	132	10000100	Ѱ	164	10100100	Д	196	11000100	д	228	11100100
...	133	10000101	Ѓ	165	10100101	Е	197	11000101	е	229	11100101
†	134	10000110	ѱ	166	10100110	Ж	198	11000110	ж	230	11100110
‡	135	10000111	Š	167	10100111	З	199	11000111	з	231	11100111
€	136	10001000	Ђ	168	10101000	И	200	11001000	и	232	11101000
‰	137	10001001	©	169	10101001	Й	201	11001001	й	233	11101001
Љ	138	10001010	€	170	10101010	К	202	11001010	к	234	11101010
«	139	10001011	«	171	10101011	Л	203	11001011	л	235	11101011
Њ	140	10001100	¬	172	10101100	М	204	11001100	м	236	11101100
Ќ	141	10001101	•	173	10101101	Н	205	11001101	н	237	11101101
Њ	142	10001110	®	174	10101110	О	206	11001110	о	238	11101110
Џ	143	10001111	Ї	175	10101111	П	207	11001111	п	239	11101111
ђ	144	10010000	°	176	10110000	Р	208	11010000	р	240	11110000
‘	145	10010001	±	177	10110001	С	209	11010001	с	241	11110001
’	146	10010010	І	178	10110010	Т	210	11010010	т	242	11110010
“	147	10010011	і	179	10110011	У	211	11010011	у	243	11110011
”	148	10010100	г	180	10110100	Ф	212	11010100	ф	244	11110100
•	149	10010101	μ	181	10110101	Х	213	11010101	х	245	11110101
—	150	10010110	¶	182	10110110	Ц	214	11010110	ц	246	11110110
—	151	10010111	·	183	10110111	Ч	215	11010111	ч	247	11110111
□	152	10011000	ё	184	10111000	Ш	216	11011000	ш	248	11111000
™	153	10011001	№	185	10111001	Щ	217	11011001	щ	249	11111001
љ	154	10011010	ё	186	10111010	Ъ	218	11011010	ъ	250	11111010
›	155	10011011	»	187	10111011	Ы	219	11011011	ы	251	11111011
њ	156	10011100	j	188	10111100	Ь	220	11011100	ь	252	11111100
ќ	157	10011101	š	189	10111101	Э	221	11011101	э	253	11111101
ћ	158	10011110	s	190	10111110	Ю	222	11011110	ю	254	11111110
џ	159	10011111	ı	191	10111111	Я	223	11011111	я	255	11111111



Unicode

Это глобальный стандарт, который присваивает каждому символу уникальный числовой код, называемый кодовой точкой.

Категории символов в Unicode



Алфавиты различных языков



Математические и технические символы



Символы для форматирования текста



Эмодзи



Диакритические знаки и другие спецсимволы



Unicode

Таблицу символов Unicode можно посмотреть на сайте

<https://symbl.cc/ru/unicode-table/>



UTF-8

Это одна из самых популярных и широко используемых кодировок для представления символов в Unicode.

Как работает кодировка UTF-8



1 байт: Символы ASCII (от 0 до 127), например, латинские буквы, цифры и некоторые знаки пунктуации.



2 байта: Символы с кодами от 128 до 2047, например, символы латиницы с диакритическими знаками (акценты), кириллические буквы и греческий алфавит.



3 байта: Символы с кодами от 2048 до 65535, включающие китайские иероглифы, японские кандзи и хангиль.



4 байта: Символы с кодами выше 65536, такие как редкие символы и эмодзи.

Преимущества UTF-8:

Эффективное использование
памяти

Для текстов на латинице (английский, испанский и т.д.) UTF-8 экономит память, используя только 1 байт на символ.

Для текстов на других языках, таких как русский или китайский, UTF-8 остаётся эффективной, занимая меньше памяти по сравнению с UTF-16 или UTF-32.

Гибкость

UTF-8 может кодировать символы любой сложности: от простых букв латинского алфавита до редких символов и эмодзи.

Международный стандарт

UTF-8 стал основным стандартом кодирования текстов в Интернете. Большинство веб-страниц и веб-приложений используют UTF-8.

Экспресс-опрос

?

Что такое функции ord, chr?



Функции `ord`, `chr`

Это встроенные функции Python, которые работают с символами и их кодировкой.



Функция `ord()`

Принимает символ и возвращает его код Unicode. Это полезно, когда нужно узнать числовое значение символа.

Пример



```
print(ord('A'))
```

```
print(ord('a'))
```

```
print(ord(' '))
```



Функция chr()

Принимает число, представляющее код Unicode, и возвращает соответствующий символ.

Пример



```
print(chr(65))
```

```
print(chr(97))
```

```
print(chr(32))
```

Экспресс-опрос

?

Для чего необходимо сравнение строк?



Сравнение строк

В Python строки можно сравнивать с помощью операторов сравнения, таких как `==`, `!=`, `<`, `>`, `<=` и `>=`.

Оператор ==



Действие

- Проверяет равенство двух строк.

Код

```
print("apple" == "apple")  
  
print("apple" == "banana")
```

Оператор !=



Действие

- Проверяет неравенство двух строк.

Код

```
print("apple" != "ap ple")
```

Операторы <, >, <=, >=



Действие

- Сравнивают строки лексикографически.
- Строки сравниваются символ за символом на основе их Unicode кодов.

Код

```
print("apple" < "banana")    # True('a' < 'b')
print("Apple" < "banana")    # True ('A' < 'b')
print("apple" > "Apple")     # True ('a' > 'A')
```

Как происходит сравнение



Действие

- **Лексикографический порядок:** Python сравнивает строки символ за символом.
- Как только найдено различие между двумя символами, сравнение завершается.

Код

```
print("abc" < "aca") # True, так как 'b' < 'c'
```

Как происходит сравнение



Действие

- **Учёт регистра:** В Unicode строчные буквы (например, 'a') имеют больший код, чем заглавные (например, 'A').
- Поэтому заглавные буквы считаются "меньшими" по значению.

Код

```
print("apple" > "Apple") # True, так как 'a' > 'A'
```

Как происходит сравнение



Действие

- **Сравнение строк разной длины:** Когда строки разной длины сравниваются, Python продолжает сравнение, пока не закончатся символы в одной из строк.
- Строка, у которой символы закончились первой, считается "меньшей".

Код

```
print("apple" < "app") # False, так как "apple"
не короче
```

Экспресс-опрос

?

Что такое функция len?



Функция len

Используется для получения длины объектов, таких как строки и другие последовательности. Она возвращает количество элементов в объекте.

Использование функции len() для строк



```
text = "Python"  
length = len(text)  
print(length)
```

Экспресс-опрос

?

Что такое индексация строк?



Индексация

Это механизм обращения к отдельным элементам в последовательных структурах данных, таких как строки и другие.

Пример индексации



Для строк:

```
text = "Python"
```

```
print(text[0]) # Вывод: 'P' (первый символ)
```

```
print(text[2]) # Вывод: 't' (третий символ)
```

Отрицательная индексация



```
text = "Python"

print(text[-1]) # Вывод: 'n' (последний символ)

print(text[-3]) # Вывод: 'h' (третий символ с конца)
```

Ошибка при обращении по несуществующему индексу



```
text = "hello"

print(text[10]) # Ошибка, так как в строке "hello" всего 5 символов

print(text[-6]) # Ошибка, так как в строке "hello" всего 5 символов
```

Пример ошибки

```
text = "hello"
print(text[10])
```

IndexError

Traceback (most recent call last)

Cell In[1], line 2

```
1 text = "hello"
----> 2 print(text[10])
```

IndexError: string index out of range

Как избежать ошибки



Действие

- **Проверять длину строки:** Прежде чем обращаться к символу по индексу, можно убедиться, что этот индекс находится в пределах допустимой длины строки.
- Для этого используется функция `len()`, которая возвращает длину строки.

Код

```
text = "hello"
index = 5

if index < len(text):
    print(text[index])

else:
    print("Индекс вне диапазона!")
```

Экспресс-опрос



Что такое срезы?



Срезы

Это способ получения подстрок из существующей строки, используя индексы.

Полезно знать



Срезы

- **start** — индекс, с которого начинается срез (включительно).
- **end** — индекс, на котором срез заканчивается (не включительно).

Синтаксис срезов

строка[start:end]

Пример среза строки



```
text = "Python programming"
```

```
print(text[0:6])
```

Если не указывать начальный индекс, срез начнётся с начала строки



```
text = "Python programming"  
print(text[:6])
```

Если не указывать конечный индекс, срез продлится до конца строки



```
text = "Python programming"  
print(text[7:])
```

Если не указывать оба индекса, срез создаст копию всей строки



```
text = "Python programming"  
print(text[:])
```




Отрицательные индексы

Можно использовать отрицательные индексы для срезов с конца строки. Например, индекс -1 соответствует последнему символу строки.

Отрицательные индексы



```
text = "Python programming"  
print(text[-11:])
```



Порядок индексов

При указании индекса нужно учитывать что end должен быть больше чем start.

Пример



```
text = "Python programming"
print(text[-11:-4]) # -11 < -4
print(text[7:14]) # 7 < 14
```

Пример



```
text = "Python programming"  
print(text[-4:-11]) # Пустая строка  
print(text[14:7])  # Пустая строка
```



Срезы с шагом

Срезы в Python можно использовать не только индексами, но и с шагом (step), который указывает, через сколько символов выбирать элементы.

Полезно знать



Срезы

- **start** — начальный индекс (включительно).
- **end** — конечный индекс (не включительно).
- **step** — шаг, который определяет, через сколько символов брать элементы.

Синтаксис с шагом

строка[start:end:step]

Пример с положительным шагом



```
text = "Python programming"  
print(text[0:12:2]) # Вывод: 'Pto rg'
```


Пример с отрицательным шагом



```
text = "Python programming"  
print(text[12:6:-1]) # Вывод: 'gimmarg'
```

Порядок индексов при отрицательном шаге



```
text = "Python programming"
print(text[-4:-11:-1]) # -11 < -4
print(text[14:7:-1]) # 7 < 14
```

Пример



```
text = "Python programming"

print(text[-11:-4:-1]) # Пустая строка

print(text[7:14:-1]) # Пустая строка
```

Срез всей строки в обратном порядке



```
text = "Python"  
print(text[::-1]) # Вывод: 'nohtyP'
```

Экспресс-опрос

?

Что такое оператор in?



Оператор in

Используется для проверки, содержит ли строка определённый элемент или подстроку.

Пример с использованием оператора in для строк



```
text = "Python programming"  
  
print("Python" in text)  
  
print("Hello" in text)
```



Комбинация not и in

Используется для проверки, что элемент не содержится в строке или другой последовательности. not просто инвертирует результат выражения с оператором in.

Пример



```
text = "Python programming"
```

```
print("Java" not in text)
```



ЗАДАНИЕ



Задание

Завершить работу над задачами с практикума.



Задание 1

Округление вверх и вниз

Напишите программу, которая:

- принимает число с плавающей запятой
- выводит результат округления вверх и вниз

Пример вывода:

Введите число: 4.3

Округление вверх: 5

Округление вниз: 4

Задание 2



Сумма чисел

Напишите программу, которая принимает число от пользователя и выводит сумму с чисел от 1 до самого числа.

Пример вывода:

Введите число: 5

Сумма чисел от 1 до 5 = 15

Задание 3

Счётчик факториала

Напишите программу, которая рассчитывает факториал заданного числа (произведение всех целых чисел от 1 до самого числа) двумя способами. Выведите оба результата и сравните равны ли они.

Пример вывода:

Введите число: 5

Факториал числа 5 первым способом: 120

Факториал числа 5 вторым способом: 120

Результаты равны? True

Задание 4



Симулятор кассы

Напишите программу, которая принимает от пользователя цены товаров. Пользователь вводит цены по одному (ввод завершается при вводе нуля).

Программа должна:

- Рассчитать и вывести общую сумму покупки, округленную до целого числа.
- Определить, какую сумму покупатель должен заплатить, если у него есть только купюры номиналом 100 евро.
- Рассчитать и вывести сдачу, которую нужно дать покупателю.

Задание 5

Учет доходов и расходов

Напишите программу, которая принимает на вход положительные и отрицательные значения и останавливается получив ноль. Программа должна рассчитать сумму доходов (положительные) и расходов (отрицательные), а также итоговый баланс.

Пример вывода:

Введите доход, расход или '0' для получения итога: 15000

Введите доход, расход или '0' для получения итога: -5000

Введите доход, расход или '0' для получения итога: -3000

Введите доход, расход или '0' для получения итога: 0

Доходы: 15000

Расходы: 8000

Итоговый баланс: 7000

Задание 6

Последовательности Фибоначчи

Напишите программу, которая выводит первые n чисел последовательности Фибоначчи. Число n вводится пользователем.

Пример вывода:

Введите количество чисел: 7

0 1 1 2 3 5 8

Задание 7

Число Армстронга

Напишите программу, которая запрашивает у пользователя целое число и проверяет, является ли оно числом Армстронга. Число Армстронга — это число, которое равно сумме своих цифр, которые были возведены в степень, равную количеству цифр в числе.

Например $153 = 1^3 + 5^3 + 3^3 = 1 + 125 + 27$

Пример вывода:

Введите число: 153

Число 153 является числом Армстронга.

Введите число: 123

Число 123 не является числом Армстронга.

Задание 8

Угадай за 5 попыток

Напишите программу, которая генерирует случайное число от 1 до 10 и просит пользователя угадать его за 5 попыток. Если пользователь не угадывает за 5 попыток, программа выводит загаданное число.

Пример вывода:

Попробуйте угадать число от 1 до 10: 10

Неверно! У вас осталось 4 попытки.

Попробуйте угадать число от 1 до 50: 3

Неверно! У вас осталось 3 попытки.

Попробуйте угадать число от 1 до 50: 7

Поздравляем! Вы угадали число.

Задание 9



Определение простого числа

Напишите программу, которая запрашивать число у пользователя и определяет, является ли заданное число простым.

Число является простым, если оно:

- Больше 1.
- Имеет только два делителя: единицу и само себя.

Пример вывода:

Введите число: 29

Число 29 является простым.

Введите число: 15

Число 15 не является простым.

Задание 10



Упорядоченность цифр

Напишите программу, которая получает натуральное число и определяет, являются ли его цифры упорядоченными по возрастанию.

Пример вывода:

Введите число: 1347

Цифры упорядочены по возрастанию.

Введите число: 1523

Цифры не упорядочены по возрастанию.



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Разбор домашнего задания

1. Математическое округление

Напишите программу, которая принимает число с плавающей точкой и округляет его до целого числа в соответствии с правилами школьной математики, а не банковского округления. Программа должна корректно работать как с положительными, так и с отрицательными числами.

Разбор домашнего задания

```
num = float(input("Введите вещественное число: "))
```

```
if num > 0:
```

```
    rounded_num = int(num + 0.5)
```

```
else:
```

```
    rounded_num = int(num - 0.5)
```

```
print("Округленное значение:", rounded_num)
```

Разбор домашнего задания

2. Гипотенуза треугольника

Напишите программу, которая запрашивает у пользователя длины двух катетов прямоугольного треугольника и вычисляет длину гипотенузы. Гипотенуза равна квадратному корню из суммы квадратов катетов.

Разбор домашнего задания

```
import math

a = float(input("Введите длину первого катета: "))
b = float(input("Введите длину второго катета: "))
hypotenuse = math.sqrt(a**2 + b**2)

print("Длина гипотенузы:", hypotenuse)
```

Разбор домашнего задания

1. Сумма цифр числа

Напишите программу, которая вычисляет сумму цифр введённого числа.

Разбор домашнего задания

```
num = int(input("Введите число: "))
```

```
sum_digits = 0
```

```
while num > 0:
```

```
    sum_digits += num % 10
```

```
    num //= 10
```

```
print("Сумма цифр:", sum_digits)
```

Разбор домашнего задания

1. Палиндром

Напишите программу, которая запрашивает у пользователя целое число и определяет, является ли оно палиндромом. Число является палиндромом, если оно читается одинаково слева направо и справа налево.

Разбор домашнего задания

```

number = int(input("Введите число: "))

original_number = number

reversed_number = 0

while number > 0:

    digit = number % 10

    reversed_number = reversed_number * 10 + digit

    number //= 10

if original_number == reversed_number:

    print("Число", original_number, "является палиндромом.")

else:

    print("Число", original_number, "не является палиндромом.")

```

Разбор домашнего задания

1. Проверь интуицию

Напишите программу, которая генерирует случайное число от 1 до 100 включительно и дает пользователю 10 попыток, чтобы угадать это число. Программа должна подсказывать, больше или меньше загаданное число. После завершения игры программа оценивает, насколько хорошая интуиция у игрока.

Разбор домашнего задания

```
import random

secret_number = random.randint(1, 100)

attempts = 10

used_attempts = 0

print("Угадайте число от 1 до 100. У вас", attempts, "попыток.")

while attempts > 0:

    guess = int(input("Ваше предположение: "))

    used_attempts += 1

    attempts -= 1
```

Разбор домашнего задания

```
if guess < secret_number:

    print("Загаданное число больше вашего.")

elif guess > secret_number:

    print("Загаданное число меньше вашего.")

else:

    print("Поздравляем! Вы угадали число:", secret_number, "за", used_attempts, "попыток. Отличный результат!")

    break

else:

    print("К сожалению, вы не угадали число. Загаданное число было:", secret_number)
```

Разбор домашнего задания

```
# Оценка догадливости

if used_attempts == 1:

    print("Отличная интуиция! Вы угадали с первой попытки!")

elif used_attempts <= 3:

    print("Вы настоящий мастер угадывания!")

elif used_attempts <= 7:

    print("Вы справились, неплохой результат!")

else:

    print("В следующий раз вам обязательно повезёт!")
```



ВОПРОСЫ



Заключение

